

Case analysis of higher-order data

Joshua Dunfield and Brigitte Pientka

*School of Computer Science
McGill University
Montreal, Canada
{joshua,bpientka}@cs.mcgill.ca*

Abstract

We discuss coverage checking for data that is dependently typed and is defined using higher-order abstract syntax. Unlike previous work on coverage checking for closed data, we consider open data which may depend on some context. Our work may therefore provide insight into coverage checking in Twelf, and serve as a basis for coverage checking in functional languages such as Delphin and Beluga. More generally, our work is a foundation for proofs by case analysis in systems that reason about higher-order abstract syntax.

Keywords: higher-order abstract syntax, coverage checking

1 Introduction

Over the past decade, programming and reasoning with and about data structures that contain binders has received widespread attention in programming languages and automated reasoning systems. Higher-order abstract syntax (HOAS) is a simple and elegant technique for handling binders. The central idea is easily explained: instead of representing object variables explicitly, we use meta-language variables. For example, the object-level formula $\forall x. (x = 1) \supset \neg(x = 0)$ can be represented as `forall  $\lambda x$ . (eq x (Suc Zero)) imp (not (eq x Zero))`. This avoids the need to implement common and tricky machinery such as capture-avoiding substitution, renaming and fresh name generation. When we implement proofs, higher-order abstract syntax allows us to think of hypothetical derivations, i.e. derivations that depend on assumptions as higher-order functions, where the application of a substitution lemma corresponds to a function application. For example, in natural deduction (Fig. 1), the hypothetical typing derivation for implication introduction can be elegantly modeled using higher-order functions.

The power of HOAS encodings has been shown within the logical framework LF [4] and its implementation in Twelf [10]. Recently, HOAS encodings are supported in functional programming languages such as Elphin [16], Delphin [13], and Beluga [12]. In these systems, we analyze higher-order data using pattern matching and case expressions. This requires us to validate that the patterns are exhaustive. Similarly,

Numbers $N, M ::= x$	$\text{nat} : \text{type} .$
$ 0$	$\text{Zero} : \text{nat} .$
$ \text{suc } N$	$\text{Suc} : \text{nat} \rightarrow \text{nat} .$
Propositions $A ::= N = M$	$\text{o} : \text{type} .$
$ A \supset B$	$\text{eq} : \text{nat} \rightarrow \text{nat} \rightarrow \text{o} .$
$ \forall x. A$	$\text{imp} : \text{o} \rightarrow \text{o} \rightarrow \text{o} .$
	$\text{forall} : (\text{nat} \rightarrow \text{o}) \rightarrow \text{o} .$
Natural Deduction : $\Gamma \vdash \text{nd } A$	$\text{nd} : \text{o} \rightarrow \text{type} .$
$\frac{\Gamma, u : \text{nd } A \vdash \text{nd } B}{\Gamma \vdash \text{nd } A \supset B} \supset I^u$	$\text{impi} : (\text{nd } A \rightarrow \text{nd } B) \rightarrow \text{nd } (A \text{ imp } B) .$
$\frac{\Gamma \vdash \text{nd } A \supset B \quad \Gamma \vdash \text{nd } A}{\Gamma \vdash \text{nd } B} \supset E$	$\text{impe} : \text{nd } (A \text{ imp } B) \rightarrow \text{nd } A \rightarrow \text{nd } B .$
$\frac{\Gamma \vdash \text{nd } [a/x]A}{\Gamma \vdash \text{nd } (\forall x. A)} \forall I^a$	$\text{alli} : (\Pi a : \text{nat} . \text{nd } (A \text{ a})) \rightarrow \text{nd } (\text{forall } \lambda x . A \text{ x}) .$
$\frac{(u : \text{nd } A) \in \Gamma}{\Gamma \vdash \text{nd } A} \text{Hyp} \quad \frac{\Gamma \vdash (\forall x. A)}{\Gamma \vdash \text{nd } [N/x]A} \forall E$	$\text{alle} : \text{nd } (\text{forall } \lambda x . A \text{ x}) \rightarrow \text{nd } (A \text{ N}) .$

Fig. 1. Natural deduction and its HOAS encoding

proof assistants for HOAS-based reasoning that split a goal into different cases must ensure that the cases are exhaustive. This issue arises in Twelf’s induction theorem prover [14], and in systems such as Bedwyr [1] and Abella [3].

In the first-order, simply-typed setting, analyzing data by cases is straightforward. We can just consider all declared constants of a given type. To illustrate, in Figure 1 we define a simple logic with equality on numbers in the usual style of LF [4]. The cases for the proposition A are clear: they are exactly the three proposition forms listed in the grammar. However, for numbers we do not just need cases for 0 and $\text{suc } N$, but also a case for a variable x . A similar situation comes up with higher-order data, such as derivations in natural deduction. An encoding based on higher-order abstract syntax does not represent the rule Hyp explicitly. Instead, this base case will be implicit. Thus, generating all cases requires that we consider the context and its possible elements.

Our main contribution is a theoretical framework for generating an exhaustive set of cases for objects that may refer to assumptions, i.e. open objects. While systems like Twelf and Delphin implement case analysis on open objects, previous theoretical work on coverage dealt with closed objects [15] and a clean theoretical analysis has been elusive. We believe our theory is a first step toward demystifying coverage checking in Twelf, an operation that is mysterious to many users. More immediately, our work is a foundation for languages such as Beluga [12] that case-analyze open data. We prove a property of *coverage soundness* that is needed to prove progress in Beluga.

We will begin with an example in the language Beluga, which supports programming with LF encodings in a functional setting. To emphasize the issues due to open terms, we will concentrate on the simply typed setting in this example. However, our formal framework treats dependently typed terms, which makes the problem harder. The structure of types can be observed, and this makes coverage

```

rec cntVN :  $\Pi \psi : (\text{nat})^* . \text{nat}[\psi, x : \text{nat}] \rightarrow \text{int} =$ 
 $\Lambda \psi \Rightarrow \text{fn } n \Rightarrow \text{case } n \text{ of}$ 
   $\text{box}(\psi, x. x) \Rightarrow 1$ 
   $| \text{box}(\psi, x. p[\text{id}_\psi]) \Rightarrow 0$ 
   $| \text{box}(\psi, x. \text{Zero}) \Rightarrow 0$ 
   $| \text{box}(\psi, x. \text{Suc } U[\text{id}_\psi, x]) \Rightarrow \text{cntVN } [\psi] \text{ box}(\psi, x. U[\text{id}_\psi, x])$ 

rec cntV :  $\Pi \psi : (\text{nat})^* . o[\psi, x : \text{nat}] \rightarrow \text{int} =$ 
 $\Lambda \psi \Rightarrow \text{fn } f \Rightarrow \text{case } f \text{ of}$ 
   $\text{box}(\psi, x. \text{eq } U[\text{id}_\psi, x] V[\text{id}_\psi, x]) \Rightarrow \text{cntVN } [\psi] \text{ box}(\psi, x. U[\text{id}_\psi, x])$ 
   $\quad + \text{cntVN } [\psi] \text{ box}(\psi, x. V[\text{id}_\psi, x])$ 
   $| \text{box}(\psi, x. \text{imp } U[\text{id}_\psi, x] V[\text{id}_\psi, x]) \Rightarrow \text{cntV } [\psi] \text{ box}(\psi, x. U[\text{id}_\psi, x])$ 
   $\quad + \text{cntV } [\psi] \text{ box}(\psi, x. V[\text{id}_\psi, x])$ 
   $| \text{box}(\psi, x. \text{forall}(\lambda y. W[\text{id}_\psi, x, y])) \Rightarrow \text{cntV } [\psi, y : \text{nat}] \text{ box}(\psi, y, x. W[\text{id}_\psi, x, y])$ 

```

Fig. 2. Counting free variables using pattern matching and HOAS

checking undecidable, since any set of patterns will cover all terms of an empty type and emptiness is undecidable.

2 Motivation

To motivate the problem, we consider a simple program in the Beluga language [12] that counts the free occurrences of some variable x in a formula. For example, $\forall y.(x = y) \supset (\text{Suc } y = \text{Suc } x)$ has two free occurrences of x . The data language here is first-order logic with quantification over natural numbers, as defined in Figure 1, and we analyze HOAS data via pattern matching. Using this example, we then discuss in more detail the problem of coverage.

We will write two functions to solve this problem. The function `cntV` will recursively analyze formulas. When it reaches a natural number expression, it will call a second function `cntVN`. We use modal types such as $o[x : \text{nat}, y : \text{nat}]$, which describes a formula that can refer to the variables x and y of type `nat`. The formula $((\text{eq } x \ y) \text{ imp } (\text{eq } (\text{Suc } x) (\text{Suc } y)))$ has this type.

When `cntV` recursively reaches a formula with a universal quantifier, the set of free variables grows. Hence, we need to abstract over the contexts in which the formula makes sense. Context variables ψ provide this ability.

The function `cntV` (Fig. 2) takes in a context ψ of natural numbers, a formula f , and returns an integer. Just as types classify data objects and kinds classify types, we introduce *schemas* to classify contexts. In the type declaration for the function `cntV` we say that the context variable ψ has the schema $(\text{nat})^*$, meaning that ψ stands for a data-level context whose form is $x_1 : \text{nat}, \dots, x_n : \text{nat}$. We use single capital letters U, V, W for contextual variables, which are instantiated via higher-order pattern matching.

We examine the second function, `cntV`, first. It is built by a context abstraction $\Lambda \psi$ that introduces the context variable ψ and binds every occurrence of ψ in the body. Next, we introduce the computation-level variable f of type $o[\psi, x : \text{nat}]$. In the body of the function `cntV` we case-analyze objects of type $o[\psi, x : \text{nat}]$. The `box` construct separates data-level terms (data objects) from computation-level terms. Since formulas are constructed by equality `eq`, implication `imp` and quantification

`forall`, we have cases for each of these.

When we encounter an object built from a constructor `eq`, `imp`, or `forall`, we must extract the subexpression(s) underneath. Pattern variables are characterized by a closure $U[\sigma]$ consisting of a contextual variable U and a *postponed substitution* σ . As soon as we know what the contextual variable stands for, we apply the substitution σ . In the example, the postponed substitution associated with U is the identity substitution which essentially corresponds to α -renaming. We write id_ψ for the identity substitution with domain ψ . Intuitively, one may think of the substitution associated with contextual variables which occur in patterns as a list of variables which may occur in the hole. In the object $U[\text{id}_\psi]$, for example, the contextual variable U can be instantiated with any formula which either is closed (does not refer to any bound variable listed in the context ψ) or contains a bound variable from the context ψ . Since subformulas can refer to all variables in $\psi, \mathbf{x}:\text{nat}$, we write $U[\text{id}_\psi, \mathbf{x}]$.

In the first case, for `eq`, we call `cntVN` to count the occurrences of $\mathbf{x}$ in the natural numbers $U[\text{id}_\psi, \mathbf{x}]$ and $V[\text{id}_\psi, \mathbf{x}]$, explicitly passing ψ with `cntV` $[\psi]$.

The second case for `imp` is similarly structured, calling `cntV` instead of `cntVN`.

In the third case, for `box` ($\psi, \mathbf{x}. \text{forall } (\lambda y. W[\text{id}_\psi, \mathbf{x}, y])$), we analyze the quantified formula under the assumption that y is a natural number. To do this, we pass an extended context $(\psi, y:\text{nat})$ to `cntV`. The variable x appears last in `box` ($\psi, y, \mathbf{x}. \dots$), to match the argument type $\mathbf{o}[\dots, \mathbf{x}:\text{nat}]$.

The function `cntVN` counts the occurrences of a variable $\mathbf{x}$ in an object of type $\text{nat}[\psi, \mathbf{x}:\text{nat}]$, considering four cases. The first case, `box` ($\psi, \mathbf{x}. \mathbf{x}$), matches an occurrence of $\mathbf{x}$. The second case, `box` ($\psi, \mathbf{x}. \mathbf{p}[\text{id}_\psi]$), matches a variable that is not $\mathbf{x}$ and occurs in ψ . For this case, we use a *parameter variable* $\mathbf{p}$ (using a small letter to distinguish it from a meta-variable). This represents a bound data-level variable. The substitution id_ψ associated with $\mathbf{p}$ characterizes the possible instantiations of $\mathbf{p}$. The remaining cases are straightforward.

2.1 Basic idea of coverage on open data

In this paper, we provide the foundation for ensuring that case expressions which analyze elements of type $A[\Psi]$ via pattern matching cover all possible elements of this type. For example, in the function `cntVN` we ensure that the set of patterns $\{\mathbf{x}, \mathbf{p}[\text{id}_\psi], \text{Zero}, \text{Suc } U[\text{id}_\psi, \mathbf{x}]\}$ covers the type $\text{nat}[\psi, \mathbf{x}:\text{nat}]$. In `cntV`, the set $\{\text{eq } U[\text{id}_\psi, \mathbf{x}] \mathbf{V}[\text{id}_\psi, \mathbf{x}], \text{imp } U[\text{id}_\psi, \mathbf{x}] \mathbf{V}[\text{id}_\psi, \mathbf{x}], \text{forall } (\lambda y. U[\text{id}_\psi, \mathbf{x}, y])\}$ covers all elements of type $\mathbf{o}[\psi, \mathbf{x}:\text{nat}]$.

This set of patterns for covering the type $\mathbf{o}[\psi, \mathbf{x}:\text{nat}]$ is by no means the only one. Instead of explicitly counting the occurrences of $\mathbf{x}$ in a natural number of type $\text{nat}[\psi, \mathbf{x}:\text{nat}]$, we could have used higher-order pattern matching to enforce variable dependencies, refining the pattern `eq` $U[\text{id}_\psi, \mathbf{x}] \mathbf{V}[\text{id}_\psi, \mathbf{x}]$ into the four cases

$$\{\text{eq } U[\text{id}_\psi] \mathbf{V}[\text{id}_\psi], \text{eq } U[\text{id}_\psi, \mathbf{x}] \mathbf{V}[\text{id}_\psi], \text{eq } U[\text{id}_\psi] \mathbf{V}[\text{id}_\psi, \mathbf{x}], \text{eq } U[\text{id}_\psi, \mathbf{x}] \mathbf{V}[\text{id}_\psi, \mathbf{x}]\}$$

exactly distinguishing (1) $\mathbf{x}$ occurs in neither $U[\text{id}_\psi]$ nor $\mathbf{V}[\text{id}_\psi]$, (2) $\mathbf{x}$ occurs in $U[\text{id}_\psi, \mathbf{x}]$ but not in $\mathbf{V}[\text{id}_\psi]$, (3) $\mathbf{x}$ occurs in $\mathbf{V}[\text{id}_\psi, \mathbf{x}]$ but not in $U[\text{id}_\psi]$, and (4) $\mathbf{x}$ occurs in both $U[\text{id}_\psi, \mathbf{x}]$ and $\mathbf{V}[\text{id}_\psi, \mathbf{x}]$.

Atomic types	P	$::=$	$a \ M_1 \dots M_n$
Types	A, B	$::=$	$P \mid \Pi x:A.B \mid \Sigma x:A.B$
Normal terms	M, N	$::=$	$\lambda x. M \mid (M, N) \mid R$
Neutral terms	R	$::=$	$c \mid x \mid u[\sigma] \mid p[\sigma] \mid R \ N \mid \text{proj}_1 R \mid \text{proj}_2 R$
Substitutions	σ	$::=$	$\cdot \mid \sigma ; M \mid \sigma, R \mid \text{id}_\psi$
Contexts	Ψ, Φ	$::=$	$\cdot \mid \psi \mid \Psi, x:A$
Meta-contexts	Δ	$::=$	$\cdot \mid \Delta, u::A[\Psi] \mid \Delta, p::A[\Psi] \mid \Delta, s::\Psi[\Phi]$
Schema contexts	Ω	$::=$	$\cdot \mid \Omega, \psi::W$

Fig. 3. The data level

More generally, we provide a formal framework for answering the following question: Does a set of patterns cover the type $A[\Psi]$? Alternatively, our framework provides a general way of generating a set of patterns thereby providing a foundation for splitting an object of type $A[\Psi]$ into different cases. We emphasize that while we illustrate the problem in the setting of Beluga, where contexts are explicit, the problem is similar in systems such as Delphin and Twelf, where we also must generate all objects of type A in a context Ψ .

3 Background

We will concentrate here on the data level, since we are mainly interested in testing whether a set of patterns covers a given data object. For a discussion of the computation level, see [12].

We support the logical framework LF plus dependent pairs Σ . Our data layer closely follows contextual modal type theory [8], extended with parameter variables and context variables [12], and finally with Σ types. Perhaps most importantly, we formalize schemas, which classify contexts. We only characterize normal terms since only these are meaningful in the logical framework [17, 8]. This is achieved by a syntactic distinction between normal terms M and neutral terms R . The syntax guarantees that terms contain no β -redexes, and the typing rules guarantee that all well-typed terms are fully η -expanded.

We distinguish between three¹ kinds of variables (Figure 3): *Ordinary bound variables* x and y are used to represent data-level binders and are bound by λ -abstraction. These variables are declared in a context Ψ . *Contextual variables* stand for open objects, and include *meta-variables* u and v , which represent general open objects, and *parameter variables* p that can only be instantiated with an ordinary bound variable. Contextual variables are introduced in computation-level case expressions, and can be instantiated via pattern matching. They are associated with a postponed substitution σ . The intention is to apply σ as soon as we know which term the contextual variable should stand for. The domain of σ thus includes the free variables of the object the contextual variable stands for, and the type system statically guarantees this. Contextual variables are declared in a meta-level context Δ .

¹ Prior work also considered substitution variables. Although our theory extends to substitution variables, we omit them for brevity.

Our foundation supports *context variables* ψ which allow us to reason abstractly with contexts, and write recursive computations that manipulate higher-order data. Unlike some other uses of context variables [6], a context may contain at most one context variable. As types classify objects, and kinds classify types, we introduce the notion of *schemas* W that classify contexts Ψ . Context variables' schemas are given in a schema context Ω . We define schemas in Section 3.2.

Substitutions σ are built of normal terms (in $\sigma; M$) and atomic terms (in σ, R). We do not make the domain explicit, which simplifies the theoretical development and avoids having to rename the domain of a given σ . We also have a first-class notion of identity substitution id_ψ . We write $[\sigma]N$ for substitution application.

We assume that type constants and object constants are declared in a signature S as pure LF objects—data of dependent function type. We suppress the signature since it is the same throughout all derivations. As a notational convenience, we generalize pairs to n -ary tuples, writing $\text{proj}_k^\# R$ for the k th projection of R . For example, the second element of a triple is $\text{proj}_2^\# R = \text{proj}_1(\text{proj}_2 R)$.

3.1 Data-level typing

We type data-level terms using a bidirectional system:

$$\begin{array}{ll} \Omega; \Delta; \Psi \vdash M \Leftarrow A & \text{Check normal object } M \text{ against } A \\ \Omega; \Delta; \Psi \vdash R \Rightarrow A & \text{Synthesize } A \text{ for neutral object } R \\ \Omega; \Delta; \Psi \vdash \sigma \Leftarrow \Phi & \text{Check } \sigma \text{ against context } \Phi \end{array}$$

For readability, we omit Ω in the subsequent development since it is constant; we also assume that Δ and Ψ are well-formed.

We give the typing rules for data-level terms in Figure 4. We assume that data-level type constants a together with constants c have been declared in a signature. We will tacitly rename bound variables, and maintain that contexts and substitutions declare no variable more than once. Note that substitutions σ are defined only on ordinary variables x , not on modal variables u . We also require the usual conditions on bound variables. For example, in III the bound variable x must be new and cannot already occur in Ψ . This can always be achieved via α -renaming. The typing rules for neutral terms use *hereditary substitutions* $[\cdots]_A^a$ which preserve canonical forms [8]. Hereditary substitution is defined recursively, considering both the structure of the term to which the substitution is applied and the type A of the object being substituted. Due to lack of space, we relegate the details to the appendix. We omit the subscripts for readability in what follows.

Since hereditary substitution is decidable and the rules in Figure 4 are syntax-directed, data-level typing is decidable.

3.2 Context schemas

As the earlier example illustrated, contexts play an important part in programming with open data objects. In particular, any contexts that are explicitly constructed and passed will belong to a specific context *schema*. In the earlier example, the schema $(\mathbf{nat})^*$ represented contexts of the form $x_1:\mathbf{nat}, \dots, x_n:\mathbf{nat}$. But we allow

Data-level normal terms

$$\frac{\Delta; \Psi, x:A \vdash M \Leftarrow B}{\Delta; \Psi \vdash \lambda x. M \Leftarrow \Pi x:A. B} \text{III} \quad \frac{\Delta; \Psi \vdash M_1 \Leftarrow A_1 \quad \Delta; \Psi \vdash M_2 \Leftarrow [M_1/x]_{A_1}^a A_2}{\Delta; \Psi \vdash (M_1, M_2) \Leftarrow \Sigma x:A_1. A_2} \text{SI}$$

$$\frac{\Delta; \Psi \vdash R \Rightarrow P' \quad P' = P}{\Delta; \Psi \vdash R \Leftarrow P} \text{turn}$$

Data-level neutral terms

$$\frac{x:A \in \Psi}{\Delta; \Psi \vdash x \Rightarrow A} \text{var} \quad \frac{c:A \in \Sigma}{\Delta; \Psi \vdash c \Rightarrow A} \text{con} \quad \frac{u::A[\Phi] \in \Delta \quad \Delta; \Psi \vdash \sigma \Leftarrow \Phi}{\Delta; \Psi \vdash u[\sigma] \Rightarrow [\sigma]_{\Phi}^a A} \text{mvar}$$

$$\frac{p::A[\Phi] \in \Delta \quad \Delta; \Psi \vdash \sigma \Leftarrow \Phi}{\Delta; \Psi \vdash p[\sigma] \Rightarrow [\sigma]_{\Phi}^a A} \text{param} \quad \frac{\Delta; \Psi \vdash R \Rightarrow \Pi x:A. B \quad \Delta; \Psi \vdash N \Leftarrow A}{\Delta; \Psi \vdash R N \Rightarrow [N/x]_A^a B} \text{PIE}$$

$$\frac{\Delta; \Psi \vdash R \Rightarrow \Sigma x:A_1. A_2}{\Delta; \Psi \vdash \text{proj}_1 R \Rightarrow A_1} \text{SE}_1 \quad \frac{\Delta; \Psi \vdash R \Rightarrow \Sigma x:A_1. A_2}{\Delta; \Psi \vdash \text{proj}_2 R \Rightarrow [\text{proj}_1 R/x]_{A_1}^a A_2} \text{SE}_2$$

Data-level substitutions

$$\frac{\Delta; \Psi \vdash \cdot \Leftarrow \cdot \quad \Delta; \psi, \Psi \vdash \text{id}_{\psi} \Leftarrow \psi}{\Delta; \Psi \vdash \sigma \Leftarrow \Phi \quad \Delta; \Psi \vdash R \Rightarrow A' \quad [\sigma]_{\Phi}^a A = A' \quad \Delta; \Psi \vdash \sigma \Leftarrow \Phi \quad \Delta; \Psi \vdash M \Leftarrow [\sigma]_{\Phi}^a A}{\Delta; \Psi \vdash (\sigma, R) \Leftarrow (\Phi, x:A)} \quad \frac{\Delta; \Psi \vdash \sigma \Leftarrow \Phi \quad \Delta; \Psi \vdash M \Leftarrow [\sigma]_{\Phi}^a A}{\Delta; \Psi \vdash (\sigma, M) \Leftarrow (\Phi, x:A)}$$

Fig. 4. Data-level typing and substitutions

much more expressive contexts. For instance, when reasoning about natural deductions, the rule $\supset I^u$ adds an assumption of the form $u:(\text{nd } A)$ for some concrete proposition A . The inductive definition $\Gamma' ::= \cdot \mid \Gamma', x:\text{nat}, \mid \Gamma', u:(\text{nd } A)$ corresponds to the schema $(\text{nat} + (\text{all } A:\text{o. nd } A))^*$.

We use $+$ to denote a choice of possible elements in a context, and all allows us to describe an assumption for all possible propositions A . One concrete instance of this schema is $x:\text{nat}, u:\text{nd } (\text{eq } x \ x)$, which arises when describing the derivation of $\text{forall } (\lambda x. (\text{eq } x \ x) \text{ imp } (\text{eq } (\text{Suc } x) (\text{Suc } x)))$.

We give the grammar of schemas in Figure 5. Schemas are built of elements $F_1, \dots, F_n$, each of the form $\text{all } \tilde{\Theta}. \Sigma y_1:\tilde{B}_1, \dots, y_j:\tilde{B}_j. \tilde{b}$, where $\tilde{\Theta} = x_1:\tilde{C}_1, \dots, x_k:\tilde{C}_k$. In other words, for any instantiation of $\tilde{\Theta}$ (that is, any substitution for $x_1, \dots, x_k$), the element is of $\Sigma\Pi$ -type, where we first introduce some Σ s, followed by Π s, with no subsequent Σ s. This restriction makes it easier to describe the inhabitants of the type. Twelf has a similar restriction on worlds. In Beluga, computation typing [12] guarantees that contexts matching this grammar are the only contexts created during computation.

To check a context Ψ against a schema $(F_1 + \dots + F_n)$, we check that each element $x:A$ in Ψ is an instance of a schema element $F_k = \text{all } \tilde{\Theta}. \Sigma y_1:\tilde{B}_1, \dots, y_j:\tilde{B}_j. \tilde{B}$, with all variables in $\tilde{\Theta}$ instantiated such that $x:A$ is an instance of F_k . The rule in Figure 5 uses higher-order pattern matching. The judgment $A \doteq B / (\theta, \Delta)$ means that θ is substitution s.t. $\llbracket \theta \rrbracket B = A$.

Element types	$\tilde{A} ::= \Pi x:A. \tilde{A} \mid a \ N_1 \dots N_n$
Schema elements	$F ::= \text{all } x_1:\tilde{B}_1, \dots, x_k:\tilde{B}_k. \Sigma y_1:\tilde{A}_1, \dots, y_j:\tilde{A}_j. \tilde{A}$
Schemas	$W ::= (F_1 + \dots + F_n)^*$

Context Ψ checks against schema W

$$\frac{\Omega; \Delta \vdash \cdot \Leftarrow W \quad \frac{\psi::W \in \Omega \quad \text{for some } k \quad \Omega; \Delta; \Psi \vdash A \in F_k \quad \Omega; \Delta \vdash \Psi \Leftarrow (F_1 + \dots + F_n)^*}{\Omega; \Delta \vdash \Psi, x:A \Leftarrow (F_1 + \dots + F_n)^*}}{\Omega; \Delta \vdash \psi \Leftarrow W}$$

Type A is an instance of schema element $F = \text{all } \tilde{\Theta}. \Sigma \tilde{\Phi}. \tilde{B}$

$$\frac{\begin{array}{l} \tilde{\Theta} = x_1:\tilde{C}_1, \dots, x_n:\tilde{C}_n \quad \sigma = u_1[\text{id}(\Psi)]/x_1, \dots, u_n[\text{id}(\Psi)]/x_n \\ \Omega; \Delta; u_1:\tilde{C}_1[\Psi], \dots, u_n:\tilde{C}_n[\Psi]; \Psi \vdash A \doteq [\sigma]\Sigma \tilde{\Phi}. \tilde{B} \ / \ (\theta, \Delta) \end{array}}{\Omega; \Delta; \Psi \vdash A \in \text{all } \tilde{\Theta}. \Sigma \tilde{\Phi}. \tilde{B}}$$

Fig. 5. Schemas

4 Coverage checking

In this section, we present a theory for coverage checking. The task is to decide whether every closed term of type $A[\Psi]$ is an instance of at least one of a given set of patterns; in Beluga, this is the set of patterns guarding the branches of a case expression. This is undecidable in general, since any set of patterns covers all terms of an empty type, and emptiness is undecidable [5, p. 179]. The theory here is necessarily incomplete, in the sense that we cannot always show that an empty type is covered by a given set of patterns. This seems unlikely to be a practical problem for Beluga, at least.

Coquand [2] and Schürmann and Pfenning [15] described coverage checking for closed terms. A theoretical treatment of coverage over open data has been left open.

To see that a set of patterns Z (in Beluga, the guards of a case expression) covers a given type, we usually need to *split* the type into an equivalent set of more precise patterns. To see that $Z = \{\text{Zero}, \text{Suc } u\}$ covers all (closed) terms of type $\text{nat}[\cdot]$, we need to split $\text{nat}[\cdot]$ into the pattern set $Z' = \{\text{Zero}, \text{Suc } u_1\}$. Now it is obvious that Z covers $\text{nat}[\cdot]$, because Z' —the result of splitting $\text{nat}[\cdot]$ —is α -equivalent to Z .

More generally, suppose we want to check that Z covers $\text{nat}[\Psi]$. If $\Psi \neq \cdot$, we are dealing with open data, so when we split, we must consider variables as well as constructors. Suppose the type is $\text{nat}[\psi, x:\text{nat}, y:\text{o}]$, where ψ represents a context of schema $(\text{o} + \text{nat})^*$. The split then includes the constructors, parameter variables denoting the generic case for variables from ψ (one variable for each schema element), and the concrete variables x and y :

$$\overbrace{\text{Zero}, \text{Suc } u[\text{id}_\psi, x, y]}^{\text{constructors of nat}}, \overbrace{(p_1[\text{id}_\psi] : \text{o}), (p_2[\text{id}_\psi] : \text{nat})}^{\text{variables of } \psi}, \overbrace{x, y}^{x:\text{nat}, y:\text{o}}$$

Not all of the variables are actually possible: $p_1[\text{id}_\psi]$ is of type o , but we are splitting at type nat . The concrete variable y is similarly impossible. This gives the set $\{\text{Zero}, \text{Suc } u[\text{id}_\psi, x, y], p_2[\text{id}_\psi], x\}$.

For some sets Z we would also need to split Suc 's argument $u[\text{id}_\psi, x, y]$ into

its constituent constructors and variables. Decisions about when to split are not determined by our theory; such decisions are embodied in a nondeterministic choice between rules **Obj-split** and **Obj-no-split**. Our system is thus the *foundation* for a coverage checking algorithm.

After some remarks on substitutions and higher-order pattern unification, we state some key metatheoretical results, and then describe the coverage rules.

We write $\llbracket \theta \rrbracket$ for a contextual substitution substituting for u and p variables in Δ . The judgment $\Omega; \Delta' \vdash \theta \Leftarrow \Delta$ says that θ is a contextual substitution with domain Δ and range Δ' , under the schema context Ω . We write ρ as an abbreviation for (1) a context substitution on the schema context Ω , substituting for context variables ψ , and (2) a contextual substitution θ . The judgment $\Omega'; \Delta' \vdash \rho : (\Omega; \Delta)$ says that the domain of ρ is $(\Omega; \Delta)$ and its range is $\Omega'; \Delta'$. In the rules, we write data-level substitutions as $[M/x]A$. This is actually hereditary substitution, but we omit the types. See the appendix for details.

We allow higher-order patterns in the sense of Miller [7], in which instantiated meta-variables must be applied to a distinct set of bound variables. In our setting, this means that contextual variables must be associated with a substitution such as $x_{\Phi(1)}/x_1, \dots, x_{\Phi(n)}/x_n$. Matching is decidable and efficient [11]. The proof of the following theorem is a simple extension of the one in [11].

Theorem 4.1 (Soundness of higher-order pattern unification)

If P and Q are well-formed types under $\Omega; \Delta; \Psi$, and $\Omega; \Delta; \Psi \vdash Q \doteq P / (\theta, \Delta')$, then $\Omega; \Delta' \vdash \theta : \Delta$ and $\Omega; \Delta'; \llbracket \theta \rrbracket \Psi \vdash \llbracket \theta \rrbracket P = \llbracket \theta \rrbracket Q$ and θ is the most general unifier, that is, for all $\cdot; \cdot \vdash \rho : (\Omega; \Delta)$ there exists ρ' such that $\rho = \llbracket \rho' \rrbracket \theta$.

Lemma 4.2 (Object inversion) If $\cdot; \cdot; \Psi \vdash R \Leftarrow P$ and $\vdash \Psi : W$ then

- (1) $R = c \ N_1 \ \dots \ N_k$ iff $S(c) = \Pi x_1:A_1. \dots \Pi x_k:A_k. P'$ and $[\sigma]P' = P$
- (2) $R = x \ N_1 \ \dots \ N_k$ iff $(x : \Pi x_1:A_1. \dots \Pi x_k:A_k. P') \in \Psi$ and $[\sigma]P' = P$
- (3) $R = (\text{proj}_l^\# \ y) \ N_1 \ \dots \ N_k$ iff $(y : \Sigma y_1:\tilde{A}_1. \dots y_m:\tilde{A}_m. \tilde{A}_{m+1}) \in \Psi$
and $[\text{proj}_1^\# \ y/y_1, \dots, \text{proj}_l^\# \ y/y_l] \tilde{A}_{l+1} = \Pi x_1:B_1. \dots \Pi x_k:B_k. P'$
where $1 \leq l \leq m$ and $[\sigma]P' = P$

where $\sigma = N_1/x_1, \dots, N_k/x_k$.

Proof. By case analysis and inversion on the derivation of $\cdot; \cdot; \Psi \vdash R \Leftarrow P$. $\square$

4.1 Overview of coverage judgments

Given the set of guards in a case expression, Z , we assume each pattern $\zeta \in Z$ has the form $\Pi \Delta'. \text{box}(\hat{\Psi}. M) : A[\Psi']$, where Δ' gives the types of contextual variables u and p in M (which will be bound to objects and variables, respectively, when a case expression is evaluated), and $A[\Psi']$ is the type of M . Thus, the system expects that a pattern in a case expression would not simply be $\text{Suc } u[\text{id}_\psi, x]$, but $\Pi u::\text{nat}[\psi, x:\text{nat}]. \text{box}(\psi, x. \text{Suc } u[\text{id}_\psi, x]) : \text{nat}[\psi, x:\text{nat}]$. In this example, and in many situations, Δ' and $A[\Psi']$ could be omitted in the source program and reconstructed. However, a dependently-typed Δ' such as $u::(\text{nd } (\text{eq } x x))[x:\text{nat}]$ actually restricts u to match only natural-deduction proofs of $\text{eq } x x$. Similarly, a dependently-typed A can constrain the entire pattern.

The most essential coverage judgment, $\Omega; \Delta; \Psi \vdash \text{Obj}(A) \triangleright \text{COVERED-BY } Z$, means that every object of type A is matched by at least one pattern in Z . For example, if we have a derivation of $\Omega; \cdot; \psi, x:\text{nat}, y:\text{o} \vdash \text{Obj}(\text{nat}) \triangleright \text{COVERED-BY } Z$ then Z covers the type $\text{nat}[\psi, x:\text{nat}, y:\text{o}]$.

Such a derivation has subderivations of the general form $\Omega; \Delta; \Psi \vdash \text{Obj}(A) \triangleright \mathcal{J}$, which analyzes A and “continues with” $\mathcal{J}$. The earlier judgment is an instance of this: it analyzes A and then “continues with” $\text{COVERED-BY } Z$.

The splitting operation discussed earlier manifests as subderivations of $\Omega; \Delta; \Psi \vdash M : A \triangleright \mathcal{J}$. Here, M is a term that plays the role of a pattern, with free variables $u[\sigma]$. Omitting contexts for clarity, a derivation where $A = \text{nat}[\cdot]$ would look like

$$\frac{\begin{array}{c} \overbrace{\text{Zero} : \text{nat}[\cdot]}^{M_1 \quad A_1} \triangleright \mathcal{J} \\ \vdots \\ \overbrace{\text{Suc } u[\cdot] : \text{nat}[\cdot]}^{M_2 \quad A_2} \triangleright \mathcal{J} \end{array}}{\text{Obj}(\text{nat}[\cdot]) \triangleright \mathcal{J}}$$

In general, $M_1, \dots, M_n$ collectively cover all possible terms of type A . That is, the subderivations correspond to a split into n patterns. In the example, $n = 2$.

4.2 COVERED-BY: the leaves of a coverage derivation

We said that $\text{Obj}(A) \triangleright \text{COVERED-BY } Z$ means to analyze A and “continue with” $\text{COVERED-BY } Z$. So, having analyzed A , splitting as necessary, we eventually come to subderivations of $M_k : A_k \triangleright \text{COVERED-BY } Z$. These are the outermost branches of the derivation tree, and are the only places where Z is examined. Such subderivations all have the same structure: **Covered-By- Z** picks out one pattern ζ from the set Z , and then **Covered-By- ζ** checks that M_k is an instance of ζ .

$$\frac{\frac{\Omega \vdash (\Pi \Delta. \text{box}(\hat{\Psi}. M_k) : A_k[\Psi]) \doteq \zeta \quad / (\theta, \Delta)}{\Omega \vdash \Pi \Delta. \text{box}(\hat{\Psi}. M_k) : A_k[\Psi] \text{ COVERED-BY } \zeta} \text{ Covered-By-}\zeta}{\Omega; \Delta; \Psi \vdash M_k : A_k \triangleright \text{COVERED-BY } \{\dots, \zeta, \dots\}} \text{ Covered-By-}Z$$

We assume that the pattern ζ includes an explicit meta-variable context Δ' , explicit data-level names $\hat{\Psi}'$, and an explicit type $A'[\Psi']$. Thus, the $\doteq$ premise is

$$\Omega \vdash (\Pi \Delta. \text{box}(\hat{\Psi}. M_k) : A_k[\Psi]) \doteq (\Pi \Delta'. \text{box}(\hat{\Psi}'. M') : A'[\Psi']) \quad / (\theta, \Delta)$$

This says that M_k is an instance of M' realized by θ , that is, $M_k = \llbracket \theta \rrbracket M'$. If all the M_k are instances of various patterns in Z , then Z covers all inhabitants of A .

4.3 Rules deriving $\text{Obj}(A) \triangleright \mathcal{J}$

Having explained the high-level structure of coverage derivations and the details of the leaves, we can discuss the rules with conclusions of the form $\text{Obj}(A) \triangleright \mathcal{J}$. These are the four rules at the bottom of Figure 6.

If $A = \Pi x:A_1. A_2$, we use **Obj- Π** to peel off the Π and analyze A_2 . The **LAM** is

$$\boxed{\Omega \vdash \Pi\Delta.\text{box}(\hat{\Psi}.M) : A[\Psi] \text{ COVERED-BY } \zeta}$$

$$\frac{\Omega \vdash (\Pi\Delta.\text{box}(\hat{\Psi}.M) : A[\Psi]) \doteq (\Pi\Delta'.\text{box}(\hat{\Psi}'.M') : A'[\Psi']) / (\theta, \Delta)}{\Omega \vdash \Pi\Delta.\text{box}(\hat{\Psi}.M) : A[\Psi] \text{ COVERED-BY } (\Pi\Delta'.\text{box}(\hat{\Psi}'.M') : A'[\Psi'])} \text{Covered-By-}\zeta$$

$$\boxed{\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle(A > P) \triangleright \mathcal{J}}$$

$$\frac{\Omega; \Delta; \Psi \vdash Q \Rightarrow \Leftarrow P}{\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle(Q > P) \triangleright \mathcal{J}} \text{App-}\Rightarrow \Leftarrow \quad \frac{\Omega; \Delta; \Psi \vdash Q \doteq P / (\theta, \Delta') \quad \Omega; \Delta'; \llbracket \theta \rrbracket \Psi \vdash \llbracket \theta \rrbracket R : \llbracket \theta \rrbracket P \triangleright \llbracket \theta \rrbracket \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle(Q > P) \triangleright \mathcal{J}} \text{App-}\doteq$$

$$\frac{\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle M ([M/x]B > P) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash M : A \triangleright \text{NEUTRAL}\langle R \rangle(x.B > P) \triangleright \mathcal{J}}$$

$$\frac{\Omega; \Delta; \Psi \vdash \text{Obj}(A) \triangleright \text{NEUTRAL}\langle R \rangle(x.B > P) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle(\Pi x:A.B > P) \triangleright \mathcal{J}} \text{App-}\Pi$$

for $0 \leq i \leq m$:

$$\frac{\Omega; \Delta; \Psi \vdash \text{App}\langle \text{proj}_i^\# R \rangle([\text{proj}_1^\# R/x_1, \dots, \text{proj}_i^\# R/x_i] \tilde{A}_{i+1} > P) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle(\Sigma x_1:\tilde{A}_1, \dots, x_m:\tilde{A}_m.\tilde{A}_{m+1} > P) \triangleright \mathcal{J}} \text{App-}\Sigma$$

$$\boxed{\Omega; \Delta; \Psi \vdash M : A \triangleright \mathcal{J}} \quad \frac{\Omega \vdash \Pi\Delta.\text{box}(\hat{\Psi}.M) : A[\Psi] \text{ COVERED-BY } \zeta_k}{\Omega; \Delta; \Psi \vdash M : A \triangleright \text{COVERED-BY } \{\zeta_1, \dots, \zeta_n\}} \text{Covered-By-}\zeta$$

$$\frac{\Omega; \Delta; \Psi \vdash (\lambda x.M) : (\Pi x:A_1.A_2) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi, x:A_1 \vdash M : A_2 \triangleright \text{LAM} \triangleright \mathcal{J}} \quad \frac{\Omega; \Delta; \Psi \vdash (M, N) : \Sigma x:A_1.A_2 \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash N : [M/x]A_2 \triangleright \text{PAIR2}(M:A_1, x.\bullet) \triangleright \mathcal{J}}$$

$$\frac{\Omega; \Delta; \Psi \vdash \text{Obj}([M/x]A_2) \triangleright \text{PAIR2}(M:A_1, x.\bullet) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash M : A_1 \triangleright \text{PAIR1}(\bullet, x.A_2) \triangleright \mathcal{J}}$$

$$\boxed{\Omega; \Delta; \Psi \vdash \text{Obj}(A) \triangleright \mathcal{J}}$$

$$\frac{\Omega; \Delta; \Psi, x:A_1 \vdash \text{Obj}(A_2) \triangleright \text{LAM} \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{Obj}(\Pi x:A_1.A_2) \triangleright \mathcal{J}} \text{Obj-}\Pi \quad \frac{\Omega; \Delta; \Psi \vdash \text{Obj}(A_1) \triangleright \text{PAIR1}(\bullet, x.A_2) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{Obj}(\Sigma x:A_1.A_2) \triangleright \mathcal{J}} \text{Obj-}\Sigma$$

$$\frac{\Omega; \Delta; \Psi \vdash \text{MVars}(P) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{Obj}(P) \triangleright \mathcal{J}} \text{Obj-no-split}$$

$$\frac{\begin{array}{l} \Psi = \psi, x_1:\Sigma \tilde{\Psi}_1.\tilde{A}_1, \dots, x_k:\Sigma \tilde{\Psi}_k.\tilde{A}_k \\ \Omega(\psi) = F_1 + \dots + F_m \\ \Omega; \Delta; \Psi \vdash \text{PVars}\langle \psi : F_1 \rangle > P \triangleright \mathcal{J} \\ \vdots \\ \Omega; \Delta; \Psi \vdash \text{PVars}\langle \psi : F_m \rangle > P \triangleright \mathcal{J} \end{array} \quad \begin{array}{l} \Omega; \Delta; \Psi \vdash \text{App}\langle x_1 \rangle(\Sigma \tilde{\Psi}_1.\tilde{A}_1 > P) \triangleright \mathcal{J} \\ \vdots \\ \Omega; \Delta; \Psi \vdash \text{App}\langle x_k \rangle(\Sigma \tilde{\Psi}_k.\tilde{A}_k > P) \triangleright \mathcal{J} \\ \Omega; \Delta; \Psi \vdash \text{App}\langle c_1 \rangle(S(c_1) > P) \triangleright \mathcal{J} \\ \vdots \\ \Omega; \Delta; \Psi \vdash \text{App}\langle c_n \rangle(S(c_n) > P) \triangleright \mathcal{J} \end{array}}{\Omega; \Delta; \Psi \vdash \text{Obj}(P) \triangleright \mathcal{J}} \text{Obj-split}$$

Fig. 6. Coverage checking rules

added because after analyzing A_2 , we need to put back the Π and add a λ :

$$\frac{\frac{\Omega; \Delta; \Psi \vdash (\lambda x. M) : (\Pi x:A_1. A'_2) \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash M : A'_2 \triangleright \text{LAM} \triangleright \mathcal{J}}}{\vdots} \frac{\Omega; \Delta; \Psi, x:A_1 \vdash \text{Obj}(A_2) \triangleright \text{LAM} \triangleright \mathcal{J}}{\Omega; \Delta; \Psi \vdash \text{Obj}(\Pi x:A_1. A_2) \triangleright \mathcal{J}} \text{Obj-}\Pi$$

Note that since splitting A_2 may produce several patterns, we may have more sub-derivations $(\lambda x. \dots) : (\Pi x:A_1. \dots)$ than just the one shown.

If $A = \Sigma x:A_1. A_2$, rule **Obj- Σ** first analyzes A_1 and then A_2 . The rules in Figure 6 are laid out vertically, in the same order as they appear in a derivation.

For base types P , we can either not split (rule **Obj-no-split**) or split (rule **Obj-split**). The latter rule is less complicated than it may look. The point is to split into patterns $R \ N_1 \dots N_m$, where R is a parameter $p[\sigma]$ (left-hand premises), variable x (upper-right-hand premises), or constructor c (lower-right-hand premises),

The simplest of these are the premises **App** $\langle c_k \rangle (S(c_k) > P)$ for constructors c . These cover all constructors c_k , even those for base types that are incompatible with P —those will be discarded further up the derivation.

Deriving premises of the form $\Omega; \Delta; \Psi \vdash \text{App}\langle R \rangle (S(c_k) > P) \triangleright \mathcal{J}$ is somewhat involved, since we need to generate all spines (lists of arguments) $N_1 \dots N_m$. Here, the P denotes that we are constructing objects of type P . The constructor type $S(c_k)$ must have the form $\Pi x_1:A_1. \dots \Pi x_m:A_m. Q$, where Q is a base type. In deriving this, we use **App- Π** , which uses **Obj** $\langle A_1 \rangle$ to analyze A_1 , and (through **NEUTRAL**) adds the resulting inhabitants M_1 of A_1 to c_k .

Doing this for each $x_i:A_i$ yields subderivations of **App** $\langle c_k \ N_1 \dots N_m \rangle (Q > P)$, for various spines $N_1 \dots N_m$. If Q and P do not unify (written $Q \not\Rightarrow \Leftarrow P$ in rule **App- $\Rightarrow \Leftarrow$**) we have a trivial coverage derivation, but if Q and P do unify under some θ , then we can use **App- $\div$** , which has a premise $\llbracket \theta \rrbracket R : \llbracket \theta \rrbracket P \triangleright \llbracket \theta \rrbracket \mathcal{J}$.

Returning to rule **Obj-split** itself, the premises **App** $\langle x_k \rangle (B > P) \triangleright \mathcal{J}$ for variables are structurally similar to those for constructors. However, unlike $S(c_k)$, the variable type B could contain Σ s, so we use **App- Σ** to take projections out of the tuple.

The remaining premises of **Obj-split** have the form **PVars** $\langle \psi : F \rangle > P \triangleright \mathcal{J}$, characterizing the generic variable cases.

4.4 **PVars** $\langle \psi : F \rangle > P \triangleright \mathcal{J}$: Parameter variables

Exactly one rule concludes **PVars** $\langle \dots \rangle$, the rule **PVars** in Figure 7. In **PVars**, we generate a parameter variable for each schema element. We first create a meta-variable for each **all**-quantified variable in the element. For example, if $F = \text{all } A:\text{o.nd } A$, then $p[\text{id}_\psi]$ has type $\text{nd } u[\text{id}_\psi]$ where u is a (fresh) meta-variable. In general, we get the type of a parameter from the element $\text{all } \tilde{\Theta}. \Sigma \tilde{\Phi}. \tilde{A}$ by generating a substitution σ' that instantiates all variables in $\tilde{\Theta}$ with meta-variables, and applying σ' to $\Sigma \tilde{\Phi}. \tilde{A}$. Then we use the ideas for concrete variables. Again, since $[\sigma'] \Sigma \tilde{\Phi}. \tilde{A}$ is inhabited by tuples, we consider all possible projections.

$$\boxed{\Omega; \Delta; \Psi \vdash \text{PVars} \langle \psi : \text{all } \tilde{\Theta}. \Sigma \tilde{\Phi}. \tilde{A}_{j+1} \rangle > P \triangleright \mathcal{J}}$$

$$\begin{array}{l}
\tilde{\Theta} = y_1 : \tilde{B}_1, \dots, y_n : \tilde{B}_n \text{ and } \tilde{\Phi} = x_1 : \tilde{A}_1, \dots, x_j : \tilde{A}_j \\
\sigma = u_1[\text{id}_\psi]/y_1, \dots, u_n[\text{id}_\psi]/y_n \quad \Delta_\Theta = u_1 :: \tilde{B}_1[\psi], \dots, u_n :: \tilde{B}_n[\psi] \\
\text{for } 0 \leq i \leq j: \\
\sigma' = (\text{proj}_1^\# p[\text{id}_\psi])/x_1, \dots, (\text{proj}_i^\# p[\text{id}_\psi])/x_i \\
\Omega; \Delta, \Delta_\Theta, p :: [\sigma]((\Sigma \tilde{\Phi}. \tilde{A}_{j+1})[\psi]); \Psi \vdash \text{App} \langle \text{proj}_{i+1}^\# p[\text{id}_\psi] \rangle ([\sigma'][\sigma] \tilde{A}_{i+1} > P) \triangleright \mathcal{J} \\
\hline
\Omega; \Delta; \Psi \vdash \text{PVars} \langle \psi : \text{all } \tilde{\Theta}. \Sigma \tilde{\Phi}. \tilde{A}_{j+1} \rangle > P \triangleright \mathcal{J} \quad \text{PVars}
\end{array}$$

$$\boxed{\Omega; \Delta; \Psi \vdash \text{MVars}(P) \triangleright \mathcal{J}}$$

$$\begin{array}{l}
\Omega; \Delta, u :: P[\Psi_1]; \Psi \vdash (u[\text{id}(\Psi_1)] : P) \triangleright \mathcal{J} \\
\vdots \\
\text{ValidWk}(\cdot; \cdot \vdash P[\Psi]) \\
= \{\Psi_1, \dots, \Psi_n\} \quad \Omega; \Delta, u :: P[\Psi_n]; \Psi \vdash (u[\text{id}(\Psi_n)] : P) \triangleright \mathcal{J} \\
\hline
\Omega; \Delta; \Psi \vdash \text{MVars}(P) \triangleright \mathcal{J} \quad \text{MVars}
\end{array}$$

Fig. 7. Coverage checking rules (continued)

4.5 $\text{MVars}(P) \triangleright \mathcal{J}$: General case for all ground instances of P

The premise of rule **Obj-no-split** is $\text{MVars}(P)$, which is derivable only by rule **MVars** (Figure 7). This rule does not recursively analyze the given type P . Instead, it produces patterns $u[\text{id}(\Psi_k)]$, which any object of type $P[\Psi_k]$ matches.²

Simply generating $u[\text{id}(\Psi)]$ does not suffice. Recall the earlier example where we refined the pattern $\text{eq } U[\text{id}_\psi, x] \text{ V}[\text{id}_\psi, x]$ into four cases $\{\text{eq } U[\text{id}_\psi] \text{ V}[\text{id}_\psi], \text{eq } U[\text{id}_\psi, x] \text{ V}[\text{id}_\psi], \text{eq } U[\text{id}_\psi] \text{ V}[\text{id}_\psi, x], \text{eq } U[\text{id}_\psi, x] \text{ V}[\text{id}_\psi, x]\}$.

In fact, we generate all valid weakenings of Ψ . A *weakening* $\Psi' \subseteq \Psi$ has zero or more assumptions from Ψ (preserving order). These contexts are weaker because they provide less information. Not all weakenings make sense; for example, removing $x:\text{nat}$ from $(x:\text{nat}, u:(\text{eq } x \ x))$ yields $(u:(\text{eq } x \ x))$, which is dependent on an undeclared x . The *valid* weakenings $\text{ValidWk}(\Omega; \Delta \vdash A[\Psi])$ of a context Ψ with respect to a type A are those that are well-formed and make A well-formed.

4.6 Coverage soundness

Roughly, the soundness result we need is that, if $\cdot; \cdot; \Psi \vdash \text{Obj}(A) \triangleright \text{COVERED-BY } Z$, then for every M of type A there is a pattern in Z that matches M . That theorem will not be difficult once we have a key lemma, which will guarantee that if $\mathcal{D}$ derives $\text{Obj}(A) \triangleright \mathcal{J}$ then, for every ground M' of type A , there is within $\mathcal{D}$ a derivation of $M_i : A \triangleright \mathcal{J}$, where M' is an instance of M_i . Put another way, the lemma states that the illustration from Section 4.1 is accurate.

Once we have this lemma, soundness is straightforward: if $\mathcal{J} = \text{COVERED-BY } Z$, the lemma gives a subderivation $\mathcal{D}'$ of $\dots \vdash M : A \triangleright \text{COVERED-BY } Z$, and inversions bring us to the premise of **Covered-By-Z**.

To state the lemma precisely, we first observe that the judgment form $\Omega; \Delta; \Psi \vdash \text{Obj}(A) \triangleright \mathcal{J}$ allows for nonempty Ω and Δ . However, at runtime, we only have

² The operation $\text{id}(\Psi)$ unrolls Ψ . For example, $\text{id}(\psi, x:\text{nat}) = \text{id}_\psi, x$. See the appendix for details.

concrete contexts, so Ω is empty. Also, objects are ground, containing no contextual variables u and p , so Δ is empty. We can of course have a nonempty Ψ , though since Ω is empty, Ψ will contain no context variables.

Thus, the antecedent that M' has type A can be ground: $\cdot; \cdot; \llbracket \rho \rrbracket \Psi \vdash M' \Leftarrow \llbracket \rho \rrbracket A$, where Ω and Δ are grounded by $\cdot; \cdot \vdash \rho : (\Omega; \Delta)$. In addition, the domain of $\mathcal{D}'$ need not exactly match the domain of $\mathcal{D}$. In fact, the type in $\mathcal{D}'$ will be $\llbracket \theta \rrbracket A$, where θ is a substitution from Δ to Δ' . This is consistent with the intuition that types become more precise as we move into subderivations.

As we have θ from Δ to Δ' , and ρ from $(\Omega; \Delta)$ to ground $(\cdot; \cdot)$, the lemma also asserts the existence of a ρ' from $(\Omega; \Delta')$ to ground, so that $\rho = \llbracket \rho' \rrbracket \theta$.

In part (2) of the lemma, we reason correspondingly about **App** derivations.

Lemma 4.3 (Coverage Soundness)

- (1) If $\mathcal{D} :: \Omega; \Delta; \Psi \vdash \text{Obj}(A) \triangleright \mathcal{J}$ and $\cdot; \cdot; \llbracket \rho \rrbracket \Psi \vdash M' \Leftarrow \llbracket \rho \rrbracket A$ and $\cdot; \cdot \vdash \rho : (\Omega; \Delta)$ then there exist θ and M such that $\Omega; \Delta' \vdash \theta \Leftarrow \Delta$ and $\mathcal{D}' :: \Omega; \Delta'; \llbracket \theta \rrbracket \Psi \vdash M : \llbracket \theta \rrbracket A \triangleright \llbracket \theta \rrbracket \mathcal{J}$ where $\mathcal{D}' < \mathcal{D}$ and $\Omega; \Delta'; \llbracket \theta \rrbracket \Psi \vdash M \Leftarrow \llbracket \theta \rrbracket A$ and there exists ρ' s.t. $\rho = \llbracket \rho' \rrbracket \theta$ and $M' = \llbracket \rho' \rrbracket M$.
- (2) If $\mathcal{D} :: \Omega; \Delta; \Psi \vdash \text{App}(R)(\tilde{A} > P) \triangleright \mathcal{J}$ and $\Omega; \Delta; \Psi \vdash R \Rightarrow \tilde{A}$ and $\cdot; \cdot \vdash \rho : (\Omega; \Delta)$ and for all spines $N'_1, \dots, N'_n$ of some length n such that $\cdot; \cdot; \llbracket \rho \rrbracket \Psi \vdash (\llbracket \rho \rrbracket R) N'_1 \dots N'_n \Leftarrow \llbracket \rho \rrbracket P$, then $\mathcal{D}' :: \Omega; \Delta'; \llbracket \theta \rrbracket \Psi \vdash \llbracket \theta \rrbracket (R N_1 \dots N_n) : \llbracket \theta \rrbracket P \triangleright \llbracket \theta \rrbracket \mathcal{J}$ and for all i we have $\llbracket \rho \rrbracket N_i = N'_i$ and there exists ρ' s.t. $\rho = \llbracket \rho' \rrbracket \theta$.

Proof. By complete induction on the height of $\mathcal{D}$. □

Theorem 4.4 (Coverage Soundness)

If $\cdot; \cdot; \Psi \vdash M' \Leftarrow A$ and $\cdot; \cdot; \Psi \vdash \text{Obj}(A) \triangleright \text{COVERED-BY } Z$ then there exists $\zeta \in Z$ such that $\zeta = (\Pi \Delta_k. \text{box}(\hat{\Psi}. M_k) : A_k[\Psi_k])$ and $\cdot \vdash (\Pi \Delta'. \text{box}(\hat{\Psi}. M) : A[\Psi]) \doteq \zeta / (\theta_k, \Delta')$ where $M' = \llbracket \rho' \rrbracket \llbracket \theta_k \rrbracket M_k$.

Proof. By Lemma 4.3, inversion, and correctness of higher order matching. □

5 Conclusion

Most previous work on coverage checking, such as Coquand's work [2] in the setting of Agda and later refinements of this approach [5,9], dealt with closed data objects. In the setting of logical frameworks, theoretical work on coverage also concentrated on closed objects [15]. In contrast, we have presented a framework for coverage checking terms that depend on assumptions in a given context. Schemas and parameter variables allow us to analyze generic cases for all objects represented by a context variable.

We have concentrated on the Beluga language, but systems like Delphin and Twelf have to address a very similar issue. In fact, Schürmann [14] presented early work for open data in Twelf. In Twelf, contexts are characterized by world declarations. However, there is an important difference between worlds and schemas. In Twelf, to count free occurrences of a variable, we would write a relation. But there is no way to write a generic base case for all possible variables occurring in a context represented by ψ . Instead, we must introduce dynamic extensions for each

variable encountered when we traverse a binder. Thus, the world declaration not only captures the bound variables introduced when we traverse a binder, but also a base case for each binder. Consequently, some of the base cases are scattered, and world declarations tend to be more complicated than our schema declarations. It also makes world and coverage checking significantly more complicated.

Delphin has no explicit context variables and distinguishes parameters at the type level, rather than the syntax level. Nevertheless, we believe our framework could provide insights into the Delphin coverage checker [13] as well.

We plan to implement a coverage algorithm based on the ideas in this paper within the Beluga prototype.

References

- [1] D. Baelde, A. Gacek, D. Miller, G. Nadathur, and A. Tiu. The Bedwyr system for model checking over syntactic expressions. In F. Pfenning, editor, *21st Conference on Automated Deduction*, number 4603 in LNAI, pages 391–397. Springer, 2007.
- [2] T. Coquand. Pattern matching with dependent types. In *Informal Proceedings of Workshop on Types for Proofs and Programs*, pages 71–84. Dept. of Computing Science, Chalmers Univ. of Technology and Göteborg Univ., 1992.
- [3] A. Gacek, D. Miller, and G. Nadathur. Combining generic judgments with recursive definitions. Accepted to LICS 2008. Draft dated January 21, 2008.
- [4] R. Harper, F. Honsell, and G. Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, January 1993.
- [5] C. McBride. *Dependently Typed Functional Programs and Their Proofs*. PhD thesis, University of Edinburgh, 2000. Technical Report ECS-LFCS-00-419.
- [6] A. McCreight and C. Schürmann. A meta-linear logical framework. In *4th International Workshop on Logical Frameworks and Meta-Languages (LFM’04)*, 2004.
- [7] D. Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4):497–536, 1991.
- [8] A. Nanevski, F. Pfenning, and B. Pientka. Contextual modal type theory. *ACM Transactions on Computational Logic*, 2008. Accepted, to appear.
- [9] U. Norell. *Towards a practical programming language based on dependent type theory*. PhD thesis, Department of Computer Science and Engineering, Chalmers University of Technology, Sept. 2007.
- [10] F. Pfenning and C. Schürmann. System description: Twelf — a meta-logical framework for deductive systems. In H. Ganzinger, editor, *Proceedings of the 16th International Conference on Automated Deduction (CADE-16)*, pages 202–206. Springer-Verlag LNAI 1632, 1999.
- [11] B. Pientka. *Tabled higher-order logic programming*. PhD thesis, Department of Computer Science, Carnegie Mellon University, 2003. CMU-CS-03-185.
- [12] B. Pientka. A type-theoretic foundation for programming with higher-order abstract syntax and first-class substitutions. In *35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL’08)*, pages 371–382. ACM, 2008.
- [13] A. Poswolsky and C. Schürmann. Practical programming with higher-order encodings and dependent types. In *Proceedings of the 17th European Symposium on Programming (ESOP ’08)*, Mar. 2008.
- [14] C. Schürmann. *Automating the Meta Theory of Deductive Systems*. PhD thesis, Department of Computer Science, Carnegie Mellon University, 2000. CMU-CS-00-146.
- [15] C. Schürmann and F. Pfenning. A coverage checking algorithm for LF. In D. Basin and B. Wolff, editors, *Proceedings of the 16th International Conference on Theorem Proving in Higher Order Logics (TPHOLs’03)*, volume 2758 of *Lecture Notes in Computer Science*, pages 120–135, Rome, Italy, 2003. Springer.
- [16] C. Schürmann, A. Poswolsky, and J. Sarnat. The ∇ -calculus. Functional programming with higher-order encodings. In P. Urzyczyn, editor, *Proceedings of the 7th International Conference on Typed Lambda Calculi and Applications (TLCA’05)*, Nara, Japan, volume 3461 of *Lecture Notes in Computer Science*, pages 339–353. Springer, 2005.
- [17] K. Watkins, I. Cervesato, F. Pfenning, and D. Walker. A concurrent logical framework I: Judgments and properties. Technical Report CMU-CS-02-101, Department of Computer Science, Carnegie Mellon University, 2002.

A Appendix: Substitutions

A.1 Ordinary substitution

In the definition of ordinary data-level substitutions, we need to be careful because the only meaningful data-level terms are those in canonical form. To ensure that substitution preserves canonical form, we use a technique pioneered by Watkins et al. [17] and described in detail in [8]. The idea is to define *hereditary substitution* as a primitive recursive functional that always returns a canonical object.

In the formal development, it is simpler if we can stick to non-dependent types. We therefore first define type approximations α and an erasure operation $()^-$ that removes dependencies. Before applying any hereditary substitution $[M/x]_A^a(B)$ we first erase dependencies to obtain $\alpha = A^-$ and then carry out the hereditary substitution formally as $[M/x]_\alpha^a(B)$. A similar convention applies to the other forms of hereditary substitutions.

$$\text{Type approximations } \alpha, \beta ::= a \mid \alpha \rightarrow \beta \mid \alpha \times \beta$$

Types relate to type approximations via an erasure operation $()^-$ which we overload to work on types.

$$\begin{aligned} (a \ N_1 \dots N_n)^- &= a \\ (\Pi x:A \dots B)^- &= A^- \rightarrow B^- \\ (\Sigma x:A \dots B)^- &= A^- \times B^- \end{aligned}$$

We can define $[M/x]_\alpha^n(N)$, $[M/x]_\alpha^r(R)$, and $[M/x]_\alpha^s(\sigma)$ by nested induction, first on the structure of the type approximation α and second on the structure of the objects N , R and σ . In other words, we either go to a smaller type approximation (in which case the objects can become larger), or the type approximation remains the same and the objects become smaller. We define the hereditary substitution operations in Figure A.1. We write $\alpha \leq \beta$ and $\alpha < \beta$ if α occurs in β (as a proper subexpression in the latter case). If the original term is not well-typed, a hereditary substitution, though terminating, cannot always return a meaningful term. We formalize this as failure to return a result. However, on well-typed terms, hereditary substitution will always return well-typed terms. This substitution operation can be extended to types for which we write $[M/x]_\alpha^a(A)$.

Theorem A.1 (Termination)

$[M/x]_\alpha^*(-)$ where $*$ = $\{n, r, s, a\}$ terminates, either by returning a result or failing after a finite number of steps.

Theorem A.2 (Hereditary Substitution Principles)

If $\Delta; \Psi \vdash M \Leftarrow A$ and $\Delta; \Psi, x:A, \Psi' \vdash J$ then $\Delta; \Psi, [M/x]_\alpha^* \Psi' \vdash [M/x]_\alpha^*(J)$ for all $*$ $\in \{n, r, s, a\}$.

Building on the ideas in [8], we can also define simultaneous substitution $[\sigma]_{\bar{\psi}}^n(M)$ (respectively $[\sigma]_{\bar{\psi}}^r(R)$ and $[\sigma]_{\bar{\psi}}^s(\sigma)$). We write $\bar{\psi}$ for the context approximation of Ψ

$$\begin{array}{ll}
 [M/x]_{\alpha}^n(N) = N' & \text{Normal terms } N \\
 [M/x]_{\alpha}^r(R) = R' \text{ or } M' : \alpha' & \text{Neutral terms } R \\
 [M/x]_{\alpha}^s(\sigma) = \sigma' & \text{Substitutions } \sigma
 \end{array}$$

Data-level normal terms

$$\begin{array}{ll}
 [M/x]_{\alpha}^n(\lambda y. N) = \lambda y. N' & \text{where } N' = [M/x]_{\alpha}^n(N) \\
 & \text{choosing } y \notin \text{FV}(M), \text{ and } y \neq x \\
 [M/x]_{\alpha}^n(M_1, M_2) = (N_1, N_2) & \text{if } [M/x]_{\alpha}^n(M_1) = N_1 \text{ and } [M/x]_{\alpha}^n(M_2) = N_2 \\
 [M/x]_{\alpha}^n(R) = M' & \text{if } [M/x]_{\alpha}^r(R) = M' : \alpha' \\
 [M/x]_{\alpha}^n(R) = R' & \text{if } [M/x]_{\alpha}^r(R) = R' \\
 [M/x]_{\alpha}^n(N) \text{ fails} & \text{otherwise}
 \end{array}$$

Data-level neutral terms

$$\begin{array}{ll}
 [M/x]_{\alpha}^r(x) = M : \alpha & \\
 [M/x]_{\alpha}^r(y) = y & \text{if } y \neq x \\
 [M/x]_{\alpha}^r(u[\sigma]) = u[\sigma'] & \text{where } \sigma' = [M/x]_{\alpha}^s(\sigma) \\
 [M/x]_{\alpha}^r(p[\sigma]) = p[\sigma'] & \text{where } \sigma' = [M/x]_{\alpha}^s(\sigma) \\
 [M/x]_{\alpha}^r(R N) = R' N' & \text{where } R' = [M/x]_{\alpha}^r(R) \text{ and } N' = [M/x]_{\alpha}^n(N) \\
 [M/x]_{\alpha}^r(R N) = M'' : \beta & \text{if } [M/x]_{\alpha}^r(R) = \lambda y. M' : \alpha_1 \rightarrow \beta \\
 & \text{where } \alpha_1 \rightarrow \beta \leq \alpha \text{ and } N' = [M/x]_{\alpha}^n(N) \\
 & \text{and } M'' = [N'/y]_{\alpha_1}^n(M') \\
 [M/x]_{\alpha}^r(\text{proj}_i R) = N_i : \alpha_i & \text{if } [M/x]_{\alpha}^r(R) = (N_1, N_2) : \alpha_1 \times \alpha_2 \\
 [M/x]_{\alpha}^r(\text{proj}_i R) = \text{proj}_i R' & \text{if } [M/x]_{\alpha}^r(R) = R' \\
 [M/x]_{\alpha}^r(R) \text{ fails} & \text{otherwise}
 \end{array}$$

Data-level substitutions

$$\begin{array}{ll}
 [M/x]_{\alpha}^s(\cdot) = \cdot & \\
 [M/x]_{\alpha}^s(\sigma ; N) = (\sigma' ; N') & \text{where } \sigma' = [M/x]_{\alpha}^s(\sigma) \text{ and } N' = [M/x]_{\alpha}^n(N) \\
 [M/x]_{\alpha}^s(\sigma , R) = (\sigma' , R') & \text{if } [M/x]_{\alpha}^r(R) = R' \text{ and } \sigma' = [M/x]_{\alpha}^s(\sigma) \\
 [M/x]_{\alpha}^s(\sigma , R) = (\sigma' ; M') & \text{if } [M/x]_{\alpha}^r(R) = M' : \alpha' \text{ and } \sigma' = [M/x]_{\alpha}^s(\sigma) \\
 [M/x]_{\alpha}^s(\text{id}_{\psi}) = \text{id}_{\psi} & \\
 [M/x]_{\alpha}^s(\sigma) \text{ fails} & \text{otherwise}
 \end{array}$$

Fig. A.1. Hereditary substitution (data-level)

which is defined using the erasure operation $()^-$.

$$\begin{array}{ll}
 (\cdot)^- & = \cdot \\
 (\psi)^- & = \psi \\
 (\Psi, x:A)^- & = (\Psi)^-, x:(A)^-
 \end{array}$$

A.2 Substitution operations

The different variables (ordinary variables x , context variables ψ , and contextual variables $u[\sigma]$ and $p[\sigma]$) give rise to different substitution operations. The re-

maining substitution operations do not require any significant changes from earlier work [12,8] to handle dependent types, and we revisit them in this section.

Substitution for context variables

If we encounter a context variable ψ , we simply replace it with the context Ψ .

Data-level context

$$\begin{aligned} \llbracket \Psi/\psi \rrbracket(\cdot) &= \cdot \\ \llbracket \Psi/\psi \rrbracket(\Phi, x:A) &= (\Phi', x:A') \quad \text{if } x \notin V(\Phi'), \llbracket \Psi/\psi \rrbracket A = A', \llbracket \Psi/\psi \rrbracket \Phi = \Phi' \\ \llbracket \Psi/\psi \rrbracket(\psi) &= \Psi \\ \llbracket \Psi/\psi \rrbracket(\phi) &= \phi \quad \text{if } \phi \neq \psi \end{aligned}$$

When we apply the substitution $\llbracket \Psi/\psi \rrbracket$ to the context $\Phi, x:A$, we apply the substitution to the type A , yielding some new type A' , and to the context Φ , yielding some new context Φ' . Applying the substitution to the type A is necessary in the dependently-typed setting, since A may contain terms and in particular identity substitutions id_ψ . When we replace ψ with Ψ in the substitution id_ψ , we unfold the identity substitution. Expansion of the identity substitution is defined by the operation $\text{id}(\Psi)$ for valid contexts Ψ :

$$\begin{aligned} \text{id}(\cdot) &= \cdot \\ \text{id}(\Psi, x:A) &= \text{id}(\Psi), x \\ \text{id}(\psi) &= \text{id}_\psi \end{aligned}$$

Lemma A.3 (Unfolding identity substitution)

If $\text{id}(\Psi) = \sigma$ then $\Delta; \Psi, \Psi' \vdash \sigma \Leftarrow \Psi$.

Proof. By induction on the structure of Ψ . □

When we combine Φ' and the declaration $x:A'$ to yield a new context, we must ensure that x is not already declared in Φ' . This can always be achieved by appropriately renaming bound variable occurrences. We write $V(\Phi')$ for the set of variables declared in Φ' . The rest of the definition is mostly straightforward.

Theorem A.4 (Substitution for context variables)

If $\Omega, \psi::W, \Omega'; \Delta; \Phi \vdash J$ and $\Omega \vdash \Psi \Leftarrow W$ then $\Omega, \Omega'; \llbracket \Psi/\psi \rrbracket \Delta; \llbracket \Psi/\psi \rrbracket(\Phi) \vdash \llbracket \Psi/\psi \rrbracket J$.

Proof. By induction on the first derivation using Lemma A.3. □

Contextual substitution for contextual variables

Substitution for contextual variables is a little more difficult, but is essentially similar to the definitions in [12]. We can think of $u[\sigma]$ as a closure where, as soon as we know which term u should stand for, we can apply σ to it. The typing will ensure that the type of M and the type of u agree, i.e. we can replace u of type $A[\Psi]$ with a normal term M if M has type A in the context Ψ . Because of α -conversion, the variables substituted at different occurrences of u may differ, and we write the contextual substitution as $\llbracket \hat{\Psi}.M/u \rrbracket(N)$, $\llbracket \hat{\Psi}.M/u \rrbracket(R)$, and $\llbracket \hat{\Psi}.M/u \rrbracket(\sigma)$, where $\hat{\Psi}$ binds all free variables in M . Applying $\llbracket \hat{\Psi}.M/u \rrbracket$ to the closure $u[\sigma]$ first obtains the

simultaneous substitution $\sigma' = \llbracket \hat{\Psi}.M/u \rrbracket \sigma$, but instead of returning $M[\sigma']$, it eagerly applies σ' to M . Similar ideas apply to parameter substitutions, which are written $\llbracket \hat{\Psi}.x/p \rrbracket(M)$, $\llbracket \hat{\Psi}.x/p \rrbracket(R)$ and $\llbracket \hat{\Psi}.x/p \rrbracket(\sigma)$. Parameter substitution could not be achieved with the previous definition of contextual substitution for meta-variables, since it only allows us to substitute a normal term for a meta-variable, and x is only normal if it is of atomic type.

Finally, we use simultaneous contextual substitutions, built of either meta-variables, $(\theta, \hat{\Psi}.M/u)$, or parameter variables, $(\theta, \hat{\Psi}.x/p)$. The judgment $\Delta \vdash \theta \Leftarrow \Delta'$ checks that the contextual substitution θ maps contextual variables from Δ' to the contextual variables in Δ .

Simultaneous contextual substitution

$$\frac{}{\Delta \vdash \cdot \Leftarrow \cdot} \quad \frac{\Delta \vdash \theta \Leftarrow \Delta' \quad \Delta; \llbracket \theta \rrbracket \Psi \vdash M \Leftarrow \llbracket \theta \rrbracket A}{\Delta \vdash (\theta, \hat{\Psi}.M/u) \Leftarrow \Delta', u::A[\Psi]} \quad \frac{\Delta \vdash \theta \Leftarrow \Delta' \quad \Delta; \llbracket \theta \rrbracket \Psi \vdash x \Rightarrow A' \quad A' = \llbracket \theta \rrbracket A}{\Delta \vdash (\theta, \hat{\Psi}.x/p) \Leftarrow \Delta', p::A[\Psi]}$$

A.2.1 Contextual substitution for meta-variables

We define the contextual substitution operations for normal object, neutral objects and substitutions as follows.

$$\begin{array}{ll} \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N) = N' & \text{Normal terms } N \\ \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R) = R' \text{ or } M' : \alpha' & \text{Neutral terms } R \\ \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^s(\sigma) = \sigma' & \text{Substitutions } \sigma \end{array}$$

As mentioned earlier, $u[\sigma]$ represents a closure where, as soon as we know which term u should stand for, we can apply σ to it. Because of α -conversion, the variables substituted at different occurrences of u may differ, and we write $\hat{\Psi}.M$ to allow for necessary α -renaming. The contextual substitution is indexed with the type of u . This typing annotation is necessary since we apply the substitution σ hereditarily once we know which term u represents, and hereditary substitution requires the type to ensure termination.

Data-level normal terms

$$\begin{aligned} \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(\lambda y. N) &= \lambda y. N' \quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n N = N' \\ \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N_1, N_2) &= (N'_1, N'_2) \\ &\quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N_1) = N'_1 \text{ and } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N_2) = N'_2 \\ \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(R) &= R' \quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R) = R' \\ \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(R) &= M' \quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R) = M' : \beta \\ \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N) &\text{ fails } \quad \text{otherwise} \end{aligned}$$

In the following, note that applying $\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r$ to the closure $u[\sigma]$ first obtains the simultaneous substitution $\sigma' = \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^s \sigma$, but instead of returning $M[\sigma']$,

it eagerly applies σ' to M . However before that we recover its domain by $[\sigma'/\bar{\psi}]$. To enforce that we always return a normal object as a result of contextual substitution, we resort to ordinary hereditary substitution. For a thorough explanation, see [8].

Data-level neutral terms

$$\begin{aligned}
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(x) &= x \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(u[\sigma]) &= N : \alpha \quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \text{ and } [\sigma'/\bar{\psi}]_{\bar{\psi}}^n M = N \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(u'[\sigma]) &= u'[\sigma'] \quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \text{ choosing } u' \neq u \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(p[\sigma]) &= p[\sigma'] \quad \text{where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R \ N) &= (R' \ N') \text{ where } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r R = R' \text{ and } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N) = N' \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R \ N) &= M' : \alpha_2 \\
&\quad \text{if } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r R = \lambda x. M_0 : \alpha_1 \rightarrow \alpha_2 \text{ for } \alpha_1 \rightarrow \alpha_2 \leq \alpha[\bar{\psi}] \\
&\quad \text{and } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^n(N) = N' \text{ and } [N'/x]_{\alpha_1}^n(M_0) = M' \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(\text{proj}_i R) &= \text{proj}_i R' \text{ if } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R) = R' \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(\text{proj}_i R) &= M_i : \alpha_i \text{ if } \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R) = (M_1, M_2) : \alpha_1 \times \alpha_2 \\
\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\psi}]}^r(R) &\text{ fails } \quad \text{otherwise}
\end{aligned}$$

A.2.2 Contextual substitution for parameter variables

Contextual substitution for parameter variables follows similar principles, but substitutes an ordinary variable for a parameter variable. We write parameter substitutions as $\llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^*$ where $*$ $\in \{n, r, s, a\}$. When we encounter a parameter variable $p[\sigma]$, we replace p with the ordinary variable x and apply the substitution $\llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^s$ to σ obtaining a substitution σ' . Instead of returning a closure $x[\sigma']$ as the final result we apply σ' to the ordinary variable x . This may again yield a normal term, so we must ensure that contextual substitution for parameter variables preserves normal forms.

$$\begin{aligned}
\llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^r(p[\sigma]) &= M : \alpha \text{ if } \llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \text{ and } [\sigma'/\bar{\psi}]_{\bar{\psi}}^r x = M : \alpha \\
\llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^r(p[\sigma]) &= R \quad \text{if } \llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^s \sigma = \sigma' \text{ and } [\sigma'/\bar{\psi}]_{\bar{\psi}}^r x = R \\
\llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^r(p'[\sigma]) &= p'[\sigma'] \text{ where } \llbracket x/p \rrbracket_{\alpha[\bar{\psi}]}^s \sigma = \sigma'
\end{aligned}$$

Theorem A.5 (Termination)

$\llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\phi}]}^*(-)$ and $\llbracket \hat{\Psi}.x/p \rrbracket_{\alpha[\bar{\phi}]}^*(-)$ where $*$ $= \{n, r, s, a\}$ terminate, either by returning a result or failing after a finite number of steps.

Theorem A.6 (Contextual Substitution Principles)

- (i) If $\Delta_1; \Phi \vdash M \Leftarrow A$ and $\Delta_1, u::A[\Phi], \Delta_2; \Psi \vdash J$
then $\Delta_1, \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\phi}]}^* \Delta_2; \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\phi}]}^* \Psi \vdash \llbracket \hat{\Psi}.M/u \rrbracket_{\alpha[\bar{\phi}]}^* J$ where $*$ $= \{n, r, s, a\}$.
- (ii) If $\Delta_1; \Phi \vdash x \Rightarrow A$ and $\Delta_1, p::A[\Phi], \Delta_2; \Psi \vdash J$
then $\Delta_1, \llbracket \hat{\Psi}.x/p \rrbracket_{\alpha[\bar{\phi}]}^* \Delta_2; \llbracket \hat{\Psi}.x/p \rrbracket_{\alpha[\bar{\phi}]}^* \Psi \vdash \llbracket \hat{\Psi}.x/p \rrbracket_{\alpha[\bar{\phi}]}^* J$ where $*$ $= \{n, r, s, a\}$.